XRComm FCS 8T8R Platform

Quick Start & User Guide

XRComm Inc.

June 2026

XRComm FCS 8T8R Platform

Quick Start & User Guide

Platform	XRComm FCS 8T8R Platform
Driver Version	v1.4.3
Variant	8T8R
Audience	End users and system administrators
Classification	Proprietary

Contents

Terminology

The following terms are used throughout this document.

Term	Meaning
FlexServer	The physical host machine running the XRComm platform driver and gRPC control service. All platform binaries execute here.
Client Computer	Any machine (including the FlexServer itself) from which you run the Python gRPC control client to operate the platform.
Platform Driver	The `xrcomm-server` binary. Receives the high-speed RF stream from the NIC, manages shared memory, records to NVMe, and dispatches data to customer applications.
Control Service	The `xrcomm-grpc-ctrl` binary. Provides a gRPC interface (TCP port 50051) for remotely controlling the Platform Driver.
Customer Application	Your own software (or the provided Hello World examples) that connects to the Platform Driver via shared memory to consume IQ samples or full packets.
IQ Delivery	Mode in which interleaved int16 I/Q sample batches are dispatched to connected applications via zero-copy shared memory.
Full-Packet Delivery	Mode in which raw network packets are forwarded to a single application using DPDK secondary process attachment. Requires `sudo`.
NVMe Recording	Mode in which the received RF stream is written directly to the NVMe drive via SPDK for later offline extraction.
8T8R Playback Engine	The `xrcomm_playback` binary and `playback_grpc_server.py`. Streams pre-recorded IQ waveforms from files to the network interface at line rate with per-channel configurations.

System Topology

The platform operates as a set of background services on the FlexServer. Remote or local clients can control the platform and the playback engines over the network using Python gRPC clients.

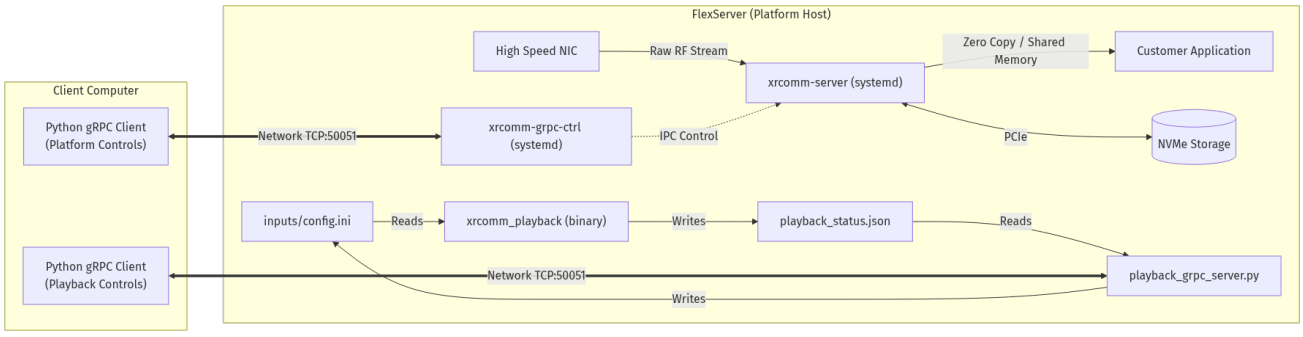

Figure 1: XRComm FCS 8T8R Platform Topology

Data flow:

1. The **High Speed NIC** receives the raw RF stream and delivers it to the **Platform Driver** via DPDK.

2. The **Platform Driver** dispatches IQ samples to connected **Customer Applications** via zero-copy shared memory.
3. Simultaneously, the Platform Driver can record the stream to the **NVMe Storage** via SPDK.
4. The **Control Service** communicates with the Platform Driver over IPC, allowing remote management.
5. The **8T8R Playback Engine** reads configuration parameters from `inputs/config.ini` and streams pre-recorded waveforms to the network.
6. The **Playback gRPC Server** writes configuration parameters to `config.ini` and reads steady-state diagnostics from `playback_status.json` dynamically.
7. Python clients on the **Client Computer** connect to both services over the network.

Package Contents

After extracting the tarball, you will find the following directory structure:

```
XRComm_FCS_platform/
  install.sh                    # Automated installer
  uninstall.sh                  # Clean uninstaller
  LICENSE                       # Software license agreement
  VERSION                       # Release version identifier
  CHANGELOG.md                  # Release notes
  8T8R_Quickstart.pdf           # This document

  XRComm_FCS_8T8R_platform/
    XRComm_platform_drivers/
      bin/                      # Precompiled platform driver binary
      include/                  # Public C headers for app development
      service/                  # Systemd unit file templates
    XRComm_platform_gRPC/
      bin/                      # Precompiled gRPC control service binary
      client/                   # Python gRPC client scripts
      proto/                    # Protobuf service definition (.proto)
      service/                  # Systemd unit file templates

  XRComm_8T8R_hello_world/
    hello_world.c               # IQ delivery example (no sudo)
    CMakeLists.txt              # Build configuration
    bin/                        # Output directory for compiled examples

  XRComm_8T8R_playback/
    bin/                        # Precompiled playback binary
    inputs/                     # Playback config and waveform files
    playback_status.json        # Live diagnostics status file
    src/                        # Playback C source code
    XRComm_platform_gRPC/
      client/                   # Python playback client scripts
      proto/                    # Playback protobuf definition (.proto)
      server/                   # Python playback gRPC server

  XRComm_NVMe_loader/
    bin/                        # Precompiled NVMe reader binary
    config/                     # Read-back config (read_config.ini)
    scripts/                    # Interactive extraction (read_nvme.sh)
```

Installation

Extract the Package

Transfer the tarball to your FlexServer and extract it:

```
tar -xzvf XRComm_FCS_platform.tar.gz
cd XRComm_FCS_platform
```

Run the Installer

```
sudo ./install.sh
```

When prompted, accept the license agreement and select **Option 2** for the 8T8R Platform.

The installer performs the following actions automatically:

Step	What it does
Dependency installation	Installs `python3-grpcio`, `python3-protobuf`, and `python3-grpc-tools` via `apt`.
Systemd service creation	Creates and enables `xrcomm-server.service` and `xrcomm-grpc-ctrl.service`.
Header file mapping	Symlinks the public C headers into `/usr/include/xrcomm-8T8R/`.
Proto compilation	Generates the Python gRPC stubs (`pipeline_control_pb2.py`) in the client directory.
Service startup	Starts both services immediately.

Uninstallation

```
sudo ./uninstall.sh
```

This stops and disables the systemd services, removes the header symlinks, and uninstalls the gRPC Python packages.

Verifying the Services

After installation, both platform services run in the background. Verify their status:

```
systemctl status xrcomm-server.service
systemctl status xrcomm-grpc-ctrl.service
```

Both should report `active (running)`.

To inspect live logs:

```
journalctl -u xrcomm-server.service -f        # Platform Driver logs
journalctl -u xrcomm-grpc-ctrl.service -f      # Control Service logs
```

To restart the services after a configuration change:

```
sudo systemctl restart xrcomm-server.service
sudo systemctl restart xrcomm-grpc-ctrl.service
```


Controlling the Platform over gRPC

The remote control client is designed to run from the **Client Computer**.

Endpoint

The Control Service listens on `0.0.0.0:50051` by default. It starts automatically as a systemd service and does not need to be launched manually.

Client Setup

The Python gRPC client can be run from **any machine** (specifically the **Client Computer**) that has network access to the FlexServer:

- **From the FlexServer itself** (via loopback `127.0.0.1`) for internal verification during installation.
- **From a remote Client Computer** for standard operational use.

Install the Python dependencies once per machine:

```
cd XRComm_FCS_8T8R_platform/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
```

The installer already generates the protobuf Python bindings on the FlexServer. If you are setting up the client on a separate machine, copy the `client/` directory there and generate the stubs:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
    --grpc_python_out=. pipeline_control.proto
```

Control Commands

Usage syntax:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 <command> [value]
```

Replace `<server-ip>` with the IP address of your FlexServer (use `127.0.0.1` when running locally).

Command	What it does
`get-flags`	Show current delivery and recording mode flags.
`set-ip on|off`	Turn IQ-sample delivery to applications on or off.
`set-dsp on|off`	Turn full-packet delivery to applications on or off.
`set-logging on|off`	Start or stop NVMe recording.
`get-stats`	Show pipeline counters and platform telemetry.
`list-ips`	List the applications currently connected to the platform.
`watch-status`	Stream live status once per second (Ctrl-C to stop).
`shutdown`	Gracefully shut the platform down.

Note: Automatic read-back capture configurations (`set-read-capture` and `set-read-rate`) are not supported over gRPC in the 8T8R platform; NVMe capture retrieval is done offline (see Section 8).

Controlling the Playback over gRPC

The 8T8R variant includes the Universal Playback Engine (`XRComm_8T8R_playback/`). The gRPC server and client are designed to control multi-channel waveform streaming.

Endpoint & Setup

The playback gRPC server binds to `[::]:50051` by default. It is run manually from the `XRComm_8T8R_playback/` directory.

Set up the client dependencies and generate the stubs on the **Client Computer**:

```
cd XRComm_8T8R_playback/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
python3 -m grpc_tools.protoc -I../proto --python_out=. \
    --grpc_python_out=. ../proto/pipeline_control.proto
```

Playback Control Commands

Use `playback_client.py` on the **Client Computer** to configure and control the playback engine:

```
python3 playback_client.py --target <server-ip>:50051 <command> [args]
```

Command	Arguments	What it does
`get-config`	None	Read all 16 current `config.ini` values (read-only snapshot)
`get`	`core_list | source_mode | sample_rate | loop_count | trigger_burst_size`	Read one scalar configuration field
`set`	`field value`	Write one scalar configuration field (e.g. `set sample_rate 1966080000`)
`get-channel`	`<channel_id>` (0..7)	Read the filename configured for the specified channel
`set-channel`	`<channel_id> <filename>`	Set the waveform file for a channel (use `""` to disable the channel)
`get-txport`	`<port_id>` (0..1)	Read the DPDK transmit port ID for a port
`set-txport`	`<port_id> <tx_port_id>`	Set the DPDK transmit port ID for a port
`get-destmac`	`<port_id>` (0..1)	Read the destination MAC address for a port
`set-destmac`	`<port_id> <mac>`	Set the destination MAC address for a port (format `XX:XX:XX:XX:XX:XX`)
`start`	None	Launch playback using the current `config.ini` parameters

Command	Arguments	What it does
`stop`	None	Cleanly stop the running playback engine
`diagnostics`	None	Take a single snapshot of the 13 steady-state diagnostic metrics
`watch`	None	Stream diagnostics once per second (Ctrl-C to stop)

Active Channels & Parameters Configuration

- **Active Channels:** Any channel with a non-empty waveform filename (e.g. `ch0_filename = tone_512.bin`) is marked active. Querying `get-config` lists all active files and enabled channels.
- **Sample Rate:** Set globally using `set sample_rate <hz>`.
- **Start Triggers:** Set interactively inside the C playback stats console (see Section 7).

Playback Console & Worked Examples

8T8R Platform Controls (Client Computer)

```
# Platform gRPC Client
C_PLAT="python3 xrcomm_grpc_client.py --host 192.168.137.217:50051"
$C_PLAT get-flags
$C_PLAT set-ip on
$C_PLAT set-logging on
```

8T8R Playback Controls (Client Computer)

```
# Playback gRPC Client
C_PLAY="python3 playback_client.py --target 192.168.137.217:50051"
$C_PLAY get-config
$C_PLAY set core_list 6,7,8,9,10
$C_PLAY set sample_rate 1966080000
$C_PLAY set-channel 0 tone_512.bin     # Enable Channel 0
$C_PLAY set-channel 1 ""               # Disable Channel 1
$C_PLAY set-destmac 0 14:FE:B5:DD:9A:82
$C_PLAY start                          # Launch streaming
$C_PLAY watch                          # Monitor stream rates
$C_PLAY stop                           # Stop streaming
```

C Console Interactive Keys (FlexServer)

When running the playback executable (`xrcomm_playback`), the console output refreshes once per second. You can input the following keys interactively:

- `+` / `-` : Adjust the tuning offset by ± 1 cycle per interval.
- `[` / `]` : Adjust the tuning offset by ± 10 cycles per interval.
- `#<val>` : Configure a new tuning rate in Hz (e.g. `#400` sets 400 Hz).
- `t<G_CH>,<offset>` : Configure a start trigger sample offset for global channel `0..7` (e.g. `t0,12` triggers CH0 at sample 12, `t3,-1` disables the trigger for CH3).
- `q` / `Q` : Gracefully stop the playback engine.

NVMe Logging and Offline Retrieval

Start and stop NVMe logging captures via the platform gRPC client:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging on   # Start logging
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging off  # Stop logging
```

The 8T8R platform records IQ samples directly to the NVMe disk via SPDK. Retrieve the captured logs offline after stopping the platform:

```
cd XRComm_NVMe_loader/scripts
sudo ./read_nvme.sh
```

Use `--first-gb=<N>` or `--last-mb=<N>` parameters on `xrcomm_nvme_reader` for partial extractions.

Troubleshooting

Symptom	What to check
Playback starts but reports files missing	Waveform files must be present inside `inputs/playback_waveforms/`.
Port binds or memory pool errors on start	Confirm the hugepages are allocated and the NIC drivers are bound (Appendix A).
Port conflict on Port 50051	If running both servers, they must listen on different ports. Use `--listen` or `--host` to map one to a different port (e.g. 50052).
Telemetry shows zero processed samples	Check that the receiver mode is active (`set-ip on` or `set-dsp on`) and the consumer application is running.

Appendix A: Environment Prerequisites

Dependency Version Check Table

Dependency	Required Version	Installation Command / Source
Ubuntu Linux	22.04 LTS or 24.04 LTS (64-bit)	Operating System standard install
DPDK	v23.11.1	`sudo apt install dpdk libdpdk-dev`
SPDK	v24.01	Build from source
gRPC & Protobuf	v1.62.0 (C++ runtime)	`sudo apt install libgrpc++-dev libprotobuf-dev`
Python 3	v3.10+	Operating System standard install
grpcio & tools	v1.62.0 (Python runtime)	`pip3 install grpcio grpcio-tools`
CMake	v3.16+	`sudo apt install cmake`
GCC	v11+	`sudo apt install build-essential`

Hugepages

```
sudo rm -rf /dev/hugepages/* /mnt/huge/*
sudo sysctl -w vm.nr_hugepages=0
sudo sysctl -w vm.nr_hugepages=112
```

Device Binding (DPDK / SPDK)

Bind the 8T8R NIC interfaces and NVMe storage drives:

```
# Bind transmit/receive NIC ports to vfio-pci
sudo modprobe vfio-pci
sudo dpdk-devbind.py --bind=vfio-pci 0000:6a:00.0 0000:6a:00.1
```


Bind NVMe drives to SPDK using the SPDK setup script.

Support: For assistance, contact XRComm Inc. using the support address provided with your delivery package.